Lecture 6

Practical

Applications of binary search using data structures such as sets, maps, multisets, and Policy-Based Data Structures (PBDS).

Dr Deepika Varshney

Policy-Based Data Structures (PBDS)

- Policy-Based Data Structures (PBDS) are advanced data structures provided by the GNU C++ STL extension. They include structures like ordered sets and maps with extra features - allowing operations like finding the k-th smallest element or the number of elements less than a given value in O(log n) time.

Headers required for using PBDS

```
#include <ext/pb_ds/assoc_container.hpp> #include <ext/pb_ds/tree_policy.hpp> using namespace __gnu_pbds;
```

- **The two most used PBDS containers are:**
- **1. Ordered Set**
- #include <ext/pb_ds/assoc_container.hpp>
- #include <ext/pb_ds/tree_policy.hpp>
- using namespace __gnu_pbds;
- template<typename T>
- using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

2. Ordered Map

```
template<typename Key, typename Value>
using ordered_map = tree<Key, Value, less<Key>, rb_tree_tag, tree_order_statistics_node_update>;
```

Operations in PBDS containers

Insertion

It adds the element to the container if it doesn't already exist.

PBDS maintains elements in sorted order automatically.

```
// Required PBDS headers
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// Ordered Set Declaration
template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

int main() {
    ordered_set<int> s;

    // Insert elements
    s.insert(10);
    s.insert(20);
    s.insert(30);

    // Print all elements
    for (int x : s)
        cout << x << " ";

    return 0;
}
```

Output

```
10 20 30
```

Output

```
10 20 30
```

Deletion

erase() removes element form the container if it exists.
No error occurs if the element is not present.

```
// Required PBDS headers
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// Ordered Set Declaration
template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

int main() {
    // Initialize the ordered set
    ordered_set<int> s;
    s.insert(10);
    s.insert(20);
    s.insert(30);

    // Erase element 20
    s.erase(20);

    // Print remaining elements
    for (int x : s)
        cout << x << " ";

    return 0;
}
```

Output

```
10 30
```

Find operation

Used to check whether an element is present.

```
// Required PBDS headers
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// Ordered Set Declaration
template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

int main() {
    // Initialize ordered_set with values
    ordered_set<int> s;
    s.insert(10);
    s.insert(20);
    s.insert(30);

    // Find an element
    if (s.find(20) != s.end())
        cout << "20 is found\n";
    else
        cout << "20 is not found\n";

    return 0;
}
```

Output

```
20 is found
```

Lower bound

lower_bound(x) returns iterator to the first element ≥ x.
Similar to lower_bound in std::set.

```
// Required PBDS headers
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// Ordered Set Declaration
template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

int main() {
    // Initialize ordered_set with values
    ordered_set<int> s;
    s.insert(10);
    s.insert(20);
    s.insert(30);

    // Find lower bound of 15
    auto it = s.lower_bound(15);
    if (it != s.end())
        cout << *it << "\n";  // Output: 20

    return 0;
}
```

Output

```
20
```

- **Number theory: Binary Exponentiation, Modular**
- **Arithmetic, Modular Inverse, Euclidean: GCD and**
- **LCM,**

Binary exponentiation is a method to compute (a raised to the power of n) using only multiplications, instead of the O(n) naïve multiplications. This significant improvement in efficiency makes it possible to calculate extremely large powers quickly, even when dealing with modular arithmetic.

The Key Idea: Binary Representation of the Exponent

Binary exponentiation uses the binary representation of the exponent to optimize power calculation. Here's how we do it:

1. Any integer exponent `n` can be written as a sum of powers of two. For example, let's express 13 in binary:
2. $13 = 1101_2 = 2^3 + 2^2 + 2^0$.

Using this, we only need to compute powers of `a` that are powers of two, like a^1, a^2, a^4, a^8.... Once we have these, we multiply the required powers in order to the set bits (1s) in the binary representation of `n`

How Binary Exponentiation Works

The key idea behind binary exponentiation is to break down the exponent into its binary representation and use the properties of exponents to simplify the calculation.

Let's break it down step by step:

1. Convert the exponent `n` to its binary representation.
2. Initialize the result as 1 and the base as `a`.
3. Iterate through each bit of the binary representation of `n` from right to left: (a). If the current bit is 1, multiply the result by the current base. (b). Square the base (multiply it by itself).
4. Return the final result.

For example, let's calculate 3^{13}:

For example, let's calculate 3^{13}:

1. Convert 13 to binary: $13_{10} = 1101_2$
2. Initialize result = 1, base = 3
3. Iterate through the bits:
   - Bit 1: result = $1 * 3 = 3$, *base* = $3 * 3 = 9$
   - Bit 0: result = 3, base = $9 * 9 = 81$
   - Bit 1: result = $3 * 81 = 243$, *base* = $81 * 81 = 6561$
   - Bit 1: result = $243 * 6561 = 1,594,323$

Thus, $3^{13} = 1,594,323$.

How Does It Work?

Let's take the example of calculating 3^{13}:

- Convert 13 to binary: 1101_2
- Break it down: $3^{13} = 3^8 * 3^4 * 3^1$

We only need to compute 3^1, 3^2, 3^4, and 3^8 and then multiply the relevant ones together:

$3^1 = 3$,

$3^2 = (3^1)^2 = 9$,

$3^4 = (3^2)^2 = 81$,

$3^8 = (3^4)^2 = 6561$

$3^{13} = 6561 * 81 * 3 = 1594323$

Complexity

The binary representation of an integer `n` has exactly $\lfloor \log_2 n \rfloor + 1$ bits.

Therefore, we only need to perform `O(log n)` multiplications to calculate `a^n`.

Recursive Binary Exponentiation (Approach 1)

The recursive approach for binary exponentiation is based on the formula:

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long

ll calpow(ll a, ll b) {
    if (b == 0)
        return 1;
    ll res = calpow(a, b / 2);
    // If we find out that b is odd, it basically does this :
    /*
        if B = 5 : b/2 = 2, i.e,( a^2 * a^2 * a ) == a ^ 5
    */
    if (b % 2)
        return res * res * a;

    else
        return res * res;
}
int main() {
    cout << calpow(3, 13) << '\n;
    // Output: 1594323
    return 0;
}
```

1. Initial Step

We start with a = 3 and n = 13.

- Since `n` is odd, we reduce the problem to 3^{12} and multiply by 3 at the end.

2. Recursive Calls Breakdown

At each recursive step, we halve the exponent `n`. The breakdown for 3^{13} is as follows:

1. **First Call:** $3^{13} = (3^6)^2 \cdot 3 = (3^6)*(3^6)*3$ (odd exponent n = 13),
2. **Second Call:** $3^6 = (3^3)^2$ (even exponent n = 6),
3. **Third Call:** $3^3 = (3^1)*(3^1)*3 = (3^1)^2*3$ (odd exponent n = 3),
4. **Fourth Call:** $3^1 = 3$ (base case where n = 1)

3. Returning from Recursion

Once we reach the base case ($3^1 = 3$), we begin to return from the recursive calls, calculating the results step by step:

1. **Fourth Call Result:** $3^1 = 3$,
2. **Third Call:** $3^3 = (3^1)*(3^1)*3 = (3^1)^2*3 = 27$,
3. **Second Call:** $3^6 = (3^3)^2 = 27^2 = 729$,
4. **First Call:** $3^{13} = (36)^2*3 = 729^2*3 = 531441*3 = 1594323$,

4. Final Result

After returning from all the recursive calls, the final result of 3^{13} is: **1594323**

> *Time complexity of this approach : O(log b)*
> *Space complexity of this approach (due to recursive call stack) : O(log b)*

Why Binary Exponentiation is Efficient

The efficiency of binary exponentiation comes from two main factors:

1. **Reduced number of multiplications**: Instead of performing `n-1` multiplications as in the naïve approach, we only perform $O(logn)$ multiplications. This is because we're essentially breaking down the problem into smaller subproblems based on the binary representation of the exponent.
2. **Reuse of previous calculations**: By squaring the base at each step, we're reusing the results of previous calculations, which significantly reduces the overall number of operations needed.

Implement pow(*x*, *n*), which calculates *x* raised to the power *n* (i.e. x^n).

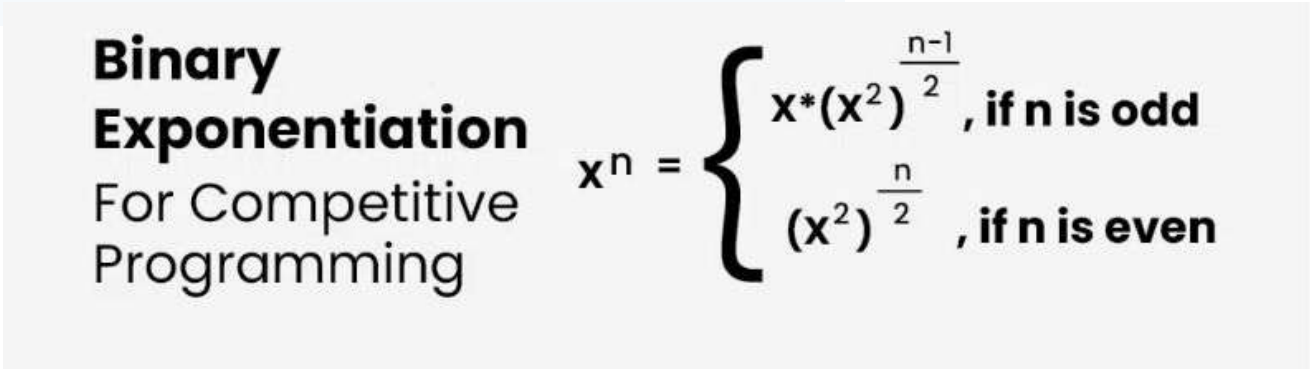
Example 1:

```
Input: x = 2.00000, n = 10
Output: 1024.00000
```

Example 2:

```
Input: x = 2.10000, n = 3
Output: 9.26100
```

In competitive programming, we often need to do a lot of big number calculations fast. Binary exponentiation is like a super shortcut for doing powers and can make programs faster. This article will show you how to use this powerful trick to enhance your coding skills.



- **What is Binary Exponentiation?**

Binary Exponentiation or Exponentiation by squaring is the process of calculating a number raised to the power another number (A^B) in Logarithmic time of the exponent or power, which speeds up the execution time of the program.

Why to Use Binary Exponentiation?

*Whenever we need to calculate (A^B), we can simple calculate the result by taking the **result** as 1 and multiplying **A** for exactly **B** times. The time complexity for this approach is O(B) and will fail when values of B in order of 10^8 or greater. This is when we can use Binary exponentiation because it can calculate the result in O(log(B)) time complexity, so we can easily calculate the results for larger values of B in order of 10^{18} or less.*

Idea Behind Binary Exponentiation:

When we are calculating (A^B), we can have 3 possible positive values of B:

- **Case 1:** If B = 0, whatever be the value of A, our result will be 1.
- **Case 2:** If B is an even number, then instead of calculating (A^B), we can calculate $((A^2)^{B/2})$ and the result will be same.
- **Case 3:** If B is an odd number, then instead of calculating (A^B), we can calculate $(A * (A^{(B-1)/2})^2)$.

Examples:

$$
\begin{aligned}
2^{12} &= (2)^{2*6} \\
&= (4)^6 \\
&= (4)^{2*3} \\
&= (16)^3 \\
&= 16 * (16)^2 \\
&= 16 * (256)^1
\end{aligned}
$$

Recursive Implementation of Binary Exponentiation:

```cpp
#include <bits/stdc++.h>
using namespace std;

long long power(long long A, long long B)
{
    if (B == 0)
        return 1;

    long long res = power(A, B / 2);

    if (B % 2)
        return res * res * A;
    else
        return res * res;
}

int main()
{
    cout << power(2, 12) << "\n";
    return 0;
}
```

Given two integers **A** and **N**, the task is to calculate **A** raised to power **N** (i.e. $\mathbf{A^N}$).

Examples:

Input: A = 3, N = 5

Output: 243

Explanation:

3 raised to power 5 = (3*3*3*3*3) = 243

Input: A = 21, N = 4

Output: 194481

Explanation:

21 raised to power 4 = (21*21*21*21) = 194481

Efficient Approach:

To optimize the above approach, the idea is to use Bit Manipulation. Convert the integer N to its binary form and follow the steps below:

Initialize ans to store the final answer of AN.

Traverse until N > 0 and in each iteration, perform Right Shift operation on it.

Also, in each iteration, multiply A with itself and update it.

If current LSB is set, then multiply current value of A to ans.

Finally, after completing the above steps, print ans.

```cpp
// C++ Program to implement
// the above approach
#include <iostream>
using namespace std;

// Function to return a^n
int powerOptimised(int a, int n)
{

    // Stores final answer
    int ans = 1;

    while (n > 0) {

        int last_bit = (n & 1);

        // Check if current LSB
        // is set
        if (last_bit) {
            ans = ans * a;
        }

        a = a * a;

        // Right shift
        n = n >> 1;
    }

    return ans;
}
```

```cpp
// Driver Code
int main()
{
    int a = 3, n = 5;

    cout << powerOptimised(a, n);

    return 0;
}
```

Output

```
243
```

Time Complexity: *O(logN)*
Auxiliary Space: *O(1)*

Another efficient approach : Recursive exponentiation

Recursive exponentiation is a method used to **efficiently** compute A^N, where A & N are integers. It leverages recursion to break down the problem into smaller subproblems. Follow the steps below :

- If N = 0, the result is always 1 because any non zero number raised to the power of 0 is 1.
- If N is even, we can compute A^N as $(A^{N/2})^2$. This is done by recursively computing $A^{N/2}$ & squaring the result.
- If N is odd, we compute A^N as $A * A^{N-1}$. Again, we recursively compute A^{N-1} & multiply the result by A.

```cpp
#include <iostream>
using namespace std;

// Function to calculate A raised to the power N using recursion
long long recursive_exponentiation(long long A, long long N) {
    if (N == 0) {
        // Base case: A^0 = 1
        return 1;
    } else if (N % 2 == 0) {
        // If N is even, recursively compute A^(N/2) and square it
        long long temp = recursive_exponentiation(A, N / 2);
        return temp * temp;
    } else {
        // If N is odd, recursively compute A^(N-1) and multiply by A
        return A * recursive_exponentiation(A, N - 1);
    }
}

int main() {
    // Test case
    cout << recursive_exponentiation(3, 5);
    return 0;
}
```

Complexity Analysis :

Time Complexity: O(logN)

Auxiliary Space: O(1)

Output

```
243
```

Problem 2: Modular Exponentiation (Power in Modular Arithmetic)

Given three integers x, n, and M, compute (x^n) % M (remainder when x raised to the power n is divided by M).

Examples :

Input: $x = 3, n = 2, M = 4$
Output: 1
Explanation: $3^2 \% 4 = 9 \% 4 = 1$.

Input: $x = 2, n = 6, M = 10$
Output: 4
Explanation: $2^6 \% 10 = 64 \% 10 = 4$.

- a = 3
- b = 13
- m = 7
- result = 1
- We repeatedly:
- If b is odd → multiply result with a
- Square a
- Divide b by 2

Stop when **b = 0**

Step	a	b	result	Operation
1	3	13 (odd)	1 → (1×3)%7 = 3	Multiply result
	(3×3)%7 = 2	6	3	Square a, b/2
2	2	6 (even)	3	Skip multiply
	(2×2)%7 = 4	3	3	Square a, b/2
3	4	3 (odd)	(3×4)%7 = 5	Multiply result
	(4×4)%7 = 2	1	5	Square a, b/2
4	2	1 (odd)	(5×2)%7 = 3	Multiply result
	(2×2)%7 = 4	0	3	Square a, b/2

[Expected Approach] Modular Exponentiation Method - O(log(n)) Time and O(1) Space

The idea of binary exponentiation is to reduce the exponent by half at each step, using squaring, which lowers the time complexity from O(n) to O(log n).
-> $x^n = (x^{n/2})^2$ if n is even.
-> $x^n = x*x^{n-1}$ if n is odd.

Step by step approach:

- Start with the result as 1.
- Use a loop that runs while the exponent n is greater than 0.
- If the current exponent is odd, multiply the result by the current base and apply the modulo.
- Square the base and take the modulo to keep the value within bounds.
- Divide the exponent by 2 (ignore the remainder).
- Repeat the process until the exponent becomes 0.

```cpp
#include<iostream>
using namespace std;

int powMod(int x, int n, int M) {
    int res = 1;

    // Loop until exponent becomes 0
    while(n >= 1) {

        // n is odd, multiply result by current x and take modulo
        if(n & 1) {
            res = (res * x) % M;

            // Reduce exponent by 1 to make it even
            n--;
        }

        // n is even, square the base and halve the exponent
        else {
            x = (x * x) % M;
            n /= 2;
        }
    }
    return res;
}

int main() {
    int x = 3, n = 2, M = 4;
    cout << powMod(x, n, M) << endl;
}
```

Output

```
1
```

Count of pairs in Array whose product is divisible by K

Given an array **A[]** and positive integer **K**, the task is to count the total number of pairs in the array whose product is divisible by **K**.

Examples :

Input: *A[] = [1, 2, 3, 4, 5], K = 2*
Output: *7*
Explanation: *The 7 pairs of indices whose corresponding products are divisible by 2 are*
(0, 1), (0, 3), (1, 2), (1, 3), (1, 4), (2, 3), and (3, 4).
Other pairs such as (0, 2) and (2, 4) have products 3 and 15 respectively, which are not divisible by 2.

Input: *A[] = [1, 2, 3, 4], K = 5*
Output: *0*
Explanation: *There does not exist any pair of indices whose corresponding product is divisible by 5.*

Given:

- An array arr[] of size n
- An integer K

Find the **number of pairs (i, j)** such that:

$$arr[i] \times arr[j] \text{ is divisible by } K$$

where $i < j$.

- we use **GCD + Number Theory** for an efficient solution.

For each element:

$$g = gcd(arr[i], K)$$

If:

$$g_1 \times g_2 \text{ is divisible by } K$$

then the pair is valid.

Step 1:

For every element, compute:

$$g = gcd(arr[i], K)$$

Store frequency of each gcd.

Step 2:

For each pair of gcd values:

If:

$$(g_1 \times g_2)\%K == 0$$

add their frequencies.

Example Dry Run

Input:

arr = [2, 3, 4, 6] K = 6

Step 1: Compute GCD with K

Element	gcd(element, 6)
2	2
3	3
4	2
6	6

Frequency Map:

2 → 2 times
3 → 1 time
6 → 1 time

Step 1: List All Possible Pairs

Possible pairs:

1.(2, 3)
2.(2, 4)
3.(2, 6)
4.(3, 4)
5.(3, 6)
6.(4, 6)

Total possible pairs = 6

Check Valid GCD Pairs

Valid Pairs Are:

- (2, 3) (2,3) → 6 % 6 = 0
- (2, 6)
- (3, 4)
- (3, 6)
- (4, 6)

Total = **5 pairs**

Efficient approach: The problem can be solved efficiently using hashing based on the following observation:

- *For checking the divisibility of product with K, better to deal with the GCD of number with **K**. This will remove all other factors from consideration. As, the number of divisors of K, would be very small when compared with the length of original array size.*
- *If **GCD(a, K) * GCD(b, K)** is divisible by **key**, then **a * b** should also be divisible by **key**.*

Follow the steps mentioned below to solve the problem:

- Create a map which will store the **GCD(A[i], K)** as **key** and its occurrence as value.
- For each element of the array, check all elements of map, whether map's element is divisible by X (X = quotient when **K** is divide by **GCD(A[i], K)**)
    - if yes add the occurrence of that element from map to answer.
    - Also, keep incrementing the occurrence for each element's **GCD** with **key**.
- Return the final count.

```cpp
// C++ program for the above approach

#include <bits/stdc++.h>
using namespace std;

// Program to count pairs whose product
// is divisible by key
long long countPairs(vector<int>& A, int
                                        key)
{
    long long ans = 0;
    unordered_map<int, int> mp;

    for (auto ele : A) {
        // Calculate gcd of nums[i] and
        // key
        long long gcd = __gcd(key, ele);
        long long x = key / gcd;

        // Iterate over all possible gcds
        for (auto it : mp)
            if (it.first % x == 0)
                // Add count to answer
                ans += it.second;
        // Add gcd to map
        mp[gcd]++;
    }

    return ans;
}

// Driver code
int main()
{
    vector<int> A = { 1, 2, 3, 4, 5 };
    int key = 2;

    cout << countPairs(A, key) << endl;
    return 0;
}
```

Output

```
7
```

Time Complexity: $O(N*K^{1/2})$
Space Complexity: $O(K^{1/2})$

914. X of a Kind in a Deck of Cards

Easy | Topics | Companies

You are given an integer array `deck` where `deck[i]` represents the number written on the `i^th` card.

Partition the cards into **one or more groups** such that:

- Each group has **exactly** `x` cards where `x > 1`, and
- All the cards in one group have the same integer written on them.

Return `true` *if such partition is possible, or* `false` *otherwise.*

Example 1:

```
Input: deck = [1,2,3,4,4,3,2,1]
Output: true
Explanation: Possible partition [1,1],[2,2],[3,3],[4,4].
```

Example 2:

```
Input: deck = [1,1,1,2,2,2,3,3]
Output: false
Explanation: No possible partition.
```

Constraints:

- `1 <= deck.length <= 10^4`
- `0 <= deck[i] < 10^4`

Solution 1: Greatest Common Divisor

First, we use an array or hash table `cnt` to count the occurrence of each number. Only when X is a divisor of the greatest common divisor of all `cnt[i]`, can it satisfy the problem's requirement.

Therefore, we find the greatest common divisor g of the occurrence of all numbers, and then check whether it is greater than or equal to 2.

The time complexity is $O(n \times \log M)$, and the space complexity is $O(n + \log M)$. Where n and M are the length of the array `deck` and the maximum value in the array `deck`, respectively.

C++

```cpp
class Solution {
public:
    bool hasGroupsSizeX(vector<int>& deck) {
        unordered_map<int, int> cnt;
        for (int x : deck) {
            ++cnt[x];
        }
        int g = cnt[deck[0]];
        for (auto& [_, x] : cnt) {
            g = gcd(g, x);
        }
        return g >= 2;
    }
};
```

Problem

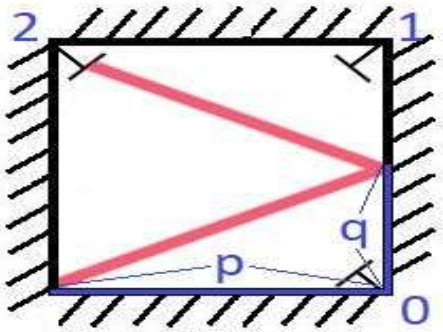
There is a special square room with mirrors on each of the four walls. Except for the southwest corner, there are receptors on each of the remaining corners, numbered `0`, `1`, and `2`.

The square room has walls of length `p` and a laser ray from the southwest corner first meets the east wall at a distance `q` from the `0th` receptor.

Given the two integers `p` and `q`, return *the number of the receptor that the ray meets first.*

The test cases are guaranteed so that the ray will meet a receptor eventually.

Example 1:



Example 2:

```
Input: p = 3, q = 1
Output: 1
```

Constraints:

- `1 <= q <= p <= 1000`

```
Input: p = 2, q = 1
Output: 2
Explanation: The ray meets receptor 2 the first time it gets reflected back to the
left wall.
```

Problem Description

You are given a square room with mirrored walls of length p. There is a laser beam that starts at the southwest corner of the room (coordinate (0, 0)) and travels at a 45-degree angle towards the opposite wall. The room has three receptors located at the following corners:

- Receptor 0 at (p, 0)
- Receptor 1 at (p, p)
- Receptor 2 at (0, p)

The laser continues to bounce off the walls until it meets one of the receptors. You are also given an integer q, the vertical distance from the 0th receptor to where the laser first meets the east wall.

Your task is to return the number of the receptor that the laser beam will meet first.

Constraints:

- 1 <= p, q <= 1000

- **Thought Process**
- At first glance, the problem seems to require simulating the path of the laser as it bounces around the room. One could imagine tracing the path step by step, keeping track of the current position and direction after each bounce. However, this brute-force approach could be inefficient, especially as p and q grow larger.

- To optimize, we need to recognize a pattern in the laser's movement. Instead of following each bounce, we can "unfold" the room: imagine extending the room by reflecting it repeatedly, so that the laser travels in a straight line through these virtual rooms. The problem then becomes finding when the laser's straight path will land on one of the receptors.

- The key insight is to find the least common multiple (LCM) of p and q, since the laser will hit a receptor when it has traveled a vertical distance that is a multiple of p and a horizontal distance that is a multiple of q.

Solution Approach

Let's break down the algorithm step by step:

1. Find the LCM of p and q:

- The laser will reach a corner when it has traveled a horizontal distance that is a multiple of p and a vertical distance that is a multiple of q.
- The first such time is their LCM.

2. Count the number of room extensions:

- Let m = LCM / p (number of room extensions horizontally).
- Let n = LCM / q (number of room extensions vertically).

3. Determine which receptor is hit:

- If m is even, the laser ends on the west wall (so receptor 2 is hit).
- If n is even, the laser ends on the south wall (so receptor 0 is hit).
- If both m and n are odd, the laser ends on the northeast corner (so receptor 1 is hit).

4. Return the receptor number based on the above logic.

This approach is efficient and avoids unnecessary simulation.

Example Walkthrough

Let's walk through an example with p = 3 and q = 1:

- LCM of 3 and 1 is 3.
- m = 3 / 3 = 1 (odd)
- n = 3 / 1 = 3 (odd)
- Both m and n are odd, so the laser hits receptor 1.

Step-by-step:

1. Laser starts at (0, 0) and moves at a 45-degree angle.

2. After 1 "room extension" horizontally (3 units), and 3 vertically (3 units), it reaches (3, 3), which is the northeast corner (receptor 1).

Time and Space Complexity

Brute-force Approach:

- Simulating each bounce would take up to O(p * q) steps in the worst case, which is inefficient for large values.
- Space complexity would be O(1), as we only need to keep track of the current position and direction.

Optimized Approach:

- Finding the LCM and performing arithmetic calculations can be done in O(log(max(p, q))) time using the Euclidean algorithm for GCD.
- Space complexity is O(1), as we only use a few variables.

```cpp
class Solution {
public:
    int mirrorReflection(int p, int q) {
        int gcd = std::__gcd(p, q);
        int lcm = p * q / gcd;
        int m = lcm / p;
        int n = lcm / q;
        if (m % 2 == 0 && n % 2 == 1) return 2;
        if (m % 2 == 1 && n % 2 == 0) return 0;
        if (m % 2 == 1 && n % 2 == 1) return 1;
        return -1; // Should not reach here
    }
};
```